

UNIVERSIDADE FEDERAL DE ITAJUBÁ

Engenharia de Dados na Prática

Exploração automatizada de dados

Profa. Bárbara Pimenta Caetano

O que veremos hoje

- 1 O que procuramos quando exploramos um dado
- 2 O que você escreveria à mão
- 3 Gerar o relatório com uma linha de código
- 4 Ler o relatório: por onde começar
- 5 Registrar os achados no README

Onde estamos



Bronze

Pronta, construída por código



Prata

Vamos preparar o terreno para ela



Ouro

Modelagem e entrega

Antes de limpar, é preciso saber exatamente o que está errado.

Já olhamos a estrutura. Falta o conteúdo

Estrutura

Já feito

O arquivo tem o formato previsto, as colunas têm os nomes esperados

Conteúdo

Hoje

Os valores dentro das colunas são confiáveis

São perguntas diferentes, e exigem ferramentas diferentes.

O que procuramos em um dado

■ Compleitude	Falta valor onde deveria haver?
■ Unicidade	Existe registro repetido?
■ Consistência	O mesmo conceito aparece escrito de formas diferentes?
■ Validade	O valor está dentro do que faz sentido?

Quatro perguntas, catorze colunas. São cinquenta e seis verificações.

O que você escreveria à mão

```
df.isna().sum()
df.duplicated().sum()
df.describe()
df.nunique()
for c in df.columns:
    print(df[c].value_counts().head())
...
```

**E ainda faltariam os gráficos, as correlações e o mapa de ausentes.
Automatizar isso não é preguiça, é padronização.**

Um detalhe do mundo real

A ferramenta que vamos usar mudou de nome duas vezes:

`pandas-profiling`

› descontinuado

`ydata-profiling`

› sem manutenção desde 2026

`fg-data-profiling`

› versão mantida hoje

**Tutorial de dois anos atrás manda instalar um pacote que hoje está parado.
Confira sempre a fonte oficial.**

PyPI, fg-data-profiling 4.19.1, abril de 2026.



Agora vamos fazer juntos

Um script curto, e depois uma leitura guiada do relatório.

PASSO 1

Instalar a ferramenta

Acrescente ao requirements.txt:

```
pandas  
kagglehub  
requests  
fg-data-profiling  
setuptools<82
```

Com o ambiente virtual ativado, instale:

```
pip install -r requirements.txt
```

O setuptools, e a versão travada, precisam de explicação. Próximo slide.

Sem isso, o import quebra

A biblioteca usa o módulo `pkg_resources`, que vive dentro do `setuptools`.

```
profile_report.py, linha 11:  
    import pkg_resources
```

O setuptools 82 removeu o `pkg_resources`. Instalar a versão mais nova não resolve: reproduz o mesmo erro.

A versão precisa ser travada:

```
pip install "setuptools<82"
```

Uma biblioteca depende de outra, que muda sozinha e quebra a primeira. É o motivo de o `requirements.txt` existir.

PASSO 2

Primeira função: achar o arquivo mais recente

Crie src/explorar.py:

```
from pathlib import Path

BRONZE = Path("dados/bronze/spotify")
PADRAO = "artists_*.csv"

def mais_recente():
    arquivos = sorted(BRONZE.glob(PADRAO))
    if not arquivos:
        raise FileNotFoundError("bronze vazia")
    return arquivos[-1]

if __name__ == "__main__":
    print(mais_recente())
```

Por que o sorted resolve

Os nomes foram criados com a data no formato ano, mês, dia:

```
artists_20260820.csv  
artists_20260917.csv  
artists_20261003.csv
```

Nesse formato, a ordem alfabética coincide com a ordem cronológica. O último da lista é sempre o mais novo.

Se tivéssemos usado dia-mês-ano, isso não funcionaria.

PASSO 3

Segunda função: gerar o relatório

```
import pandas as pd
from data_profiling import ProfileReport

RELATORIOS = Path("relatorios")

def gerar(caminho):
    df = pd.read_csv(caminho)
    perfil = ProfileReport(df, title=caminho.name)
    RELATORIOS.mkdir(exist_ok=True)
    saida = RELATORIOS / f"{caminho.stem}.html"
    perfil.to_file(saida)
    return saida
```

O import é data_profiling, e não fg_data_profiling.

PASSO 4

Juntar em main e rodar

```
def main():  
    caminho = mais_recente()  
    print("perfilando:", caminho.name)  
    print(gerar(caminho))  
  
if __name__ == "__main__":  
    main()
```

```
python src/explorar.py
```

Abra o arquivo HTML gerado no navegador.

PASSO 5

O relatório não vai para o repositório

Acrescente ao .gitignore:

```
dados/prata/*  
dados/ouro/*  
.venv/  
relatorios/
```

Bronze mais código reproduzem o relatório a qualquer momento. O que sobe é a conclusão que você tirou dele, não o arquivo.



Como ler o relatório

São muitas telas. Existe uma ordem que economiza tempo.

A ordem que funciona

- 1 Alerts** comece aqui: é a lista pronta de problemas
- 2 Overview** tamanho, duplicatas e ausentes no total
- 3 Variables** abra só as colunas que os alertas apontaram
- 4 Correlations** colunas que dizem a mesma coisa

Não leia de cima para baixo. Comece pelos alertas.

A seção de alertas

Alerts		<i>esquema da tela</i>
High correlation	Total Streams is highly correlated with Lead Streams	
Uniform	Artist Name is uniformly distributed	
Zeros	Feature Streams has 12 (2.4%) zeros	
Missing	Debut Year has 3 (0.6%) missing values	

Cada alerta é uma frase que você pode copiar para o README.

Os alertas acima são exemplos de formato. Os do seu dado podem ser outros.

Um alerta que dá para prever

Olhe os números do Drake no arquivo do Spotify:

Total	137.492,1			
Lead	94.851,1	+	Feature	42.640,9
Solo	53.323,9	+	Collab	84.168,2

■ **Total = Lead + Feature**

as duas somam o total

■ **Total = Solo + Collaborative**

outra decomposição do mesmo total

■ **% Solo + % Collab = 100**

uma é o complemento da outra

Colunas que se somam carregam informação repetida. A prata vai precisar decidir quais manter.

A seção de variáveis

Uma ficha por coluna. Abra apenas as que os alertas apontaram.

Numérica

média, mediana, mínimo, máximo, histograma

Categórica

valores mais frequentes e quantos distintos

Texto

tamanho, uso de maiúsculas, caracteres estranhos

Na ficha de texto é onde os espaços sobrando aparecem, como no Artist Type.

O que não fazer hoje

✗ **Corrigir o arquivo da bronze** ela é imutável, lembra?

✗ **Apagar coluna que pareceu inútil** isso é decisão da prata

✗ **Tentar resolver tudo de uma vez** hoje é diagnóstico, não tratamento

O que sai daqui é uma lista de problemas conhecidos, escrita em português.

PASSO 6

Rodar também na segunda fonte

Troque as duas constantes e rode de novo:

```
BRONZE = Path("dados/bronze/banco_mundial")  
PADRAO = "paises_*.csv"
```

**Se você precisa editar o script para trocar de fonte, ele ainda não está bom.
Transforme as constantes em parâmetro da função.**

Esse desconforto é o começo da modularização que veremos mais adiante.

PASSO 7

Registrar os achados no README

Amplie a seção que você criou na aula passada:

```
## Defeitos conhecidos das fontes

### Spotify (Kaggle)
- 'Artist Type' tem espaço no início do nome.
- Total, Lead e Feature são redundantes:
  Total = Lead + Feature.
- Debut Year tem N valores ausentes.

### Banco Mundial
- Devolve agregados regionais junto com os países.
- Nomes de região tem espaço no fim.
```

Uma seção por fonte, igual às pastas da bronze. Uma linha por achado.

ÚLTIMO PASSO

Publicar o que você descobriu

```
git add .  
git commit -m "Perfil das fontes e achados no README"  
git push
```

Sobe

O script explorar.py e o README atualizado

Não sobe

A pasta relatorios, que o código refaz

Na próxima aula, a lista do README vira o plano de trabalho da camada prata.